ENGG1003 WEEK 12

THE TL;DR OF SYNTAX AND THEORY FOR USING PYTHON BEYOND THIS COURSE

GITHUB: HTTPS://GITHUB.COM/KATEMCALPINE/ENGG1003

DATA TYPES

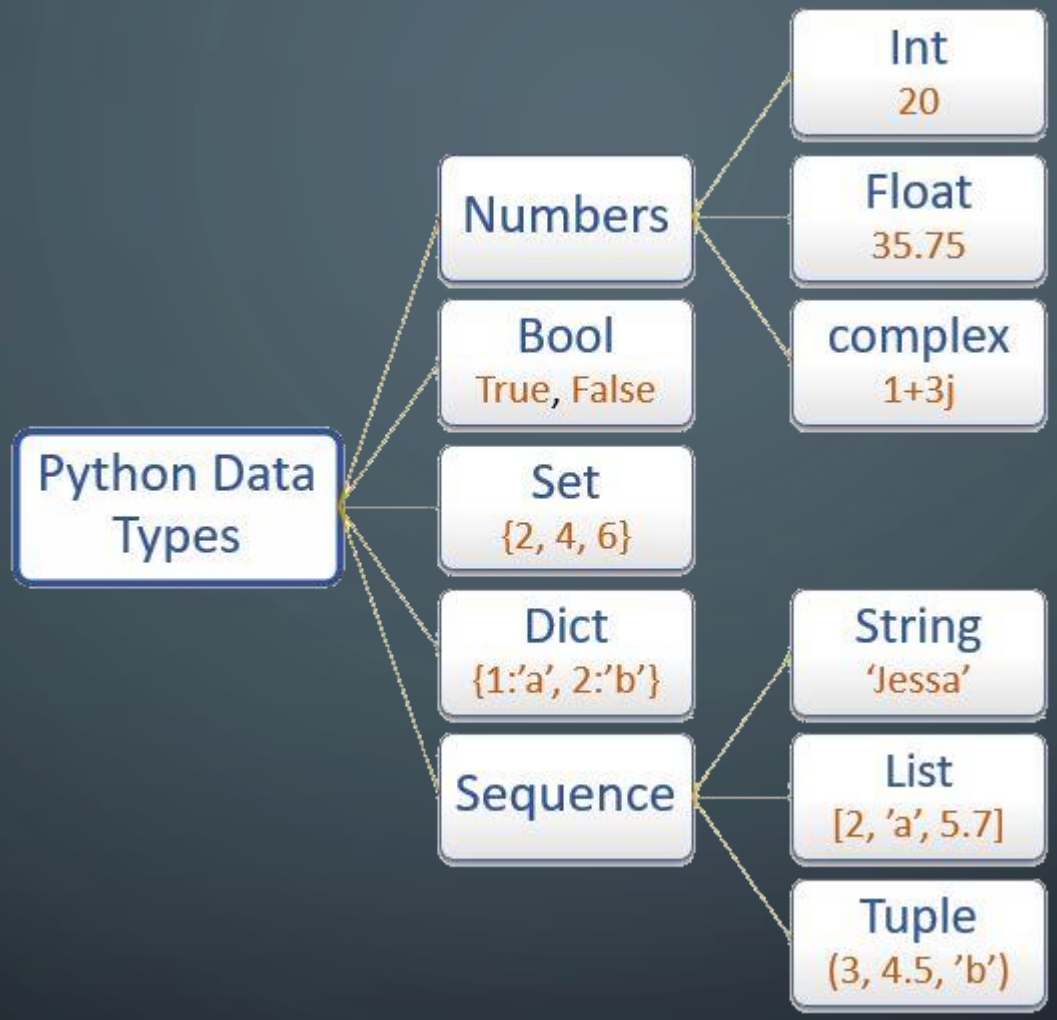


Image: https://pynative.com/python-data-types/

C Basic Data Types	32-bit CPU		64-bit CPU	
	Size (bytes)	Range	Size (bytes)	Range
char	1	-128 to 127	1	-128 to 127
short	2	-32,768 to 32,767	2	-32,768 to 32,767
int	4	-2,147,483,648 to 2,147,483,647	4	-2,147,483,648 to 2,147,483,647
long	4	-2,147,483,648 to 2,147,483,647	8	-9,223,372,036,854,775,808-9,223,372,036,854,775,807
long long	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807	8	9,223,372,036,854,775,808-9,223,372,036,854,775,807
float	4	3.4E +/- 38	4	3.4E +/- 38
double	8	1.7E +/- 308	8	1.7E +/- 308

Image: https://www.techbeamers.com/c-datatypes/

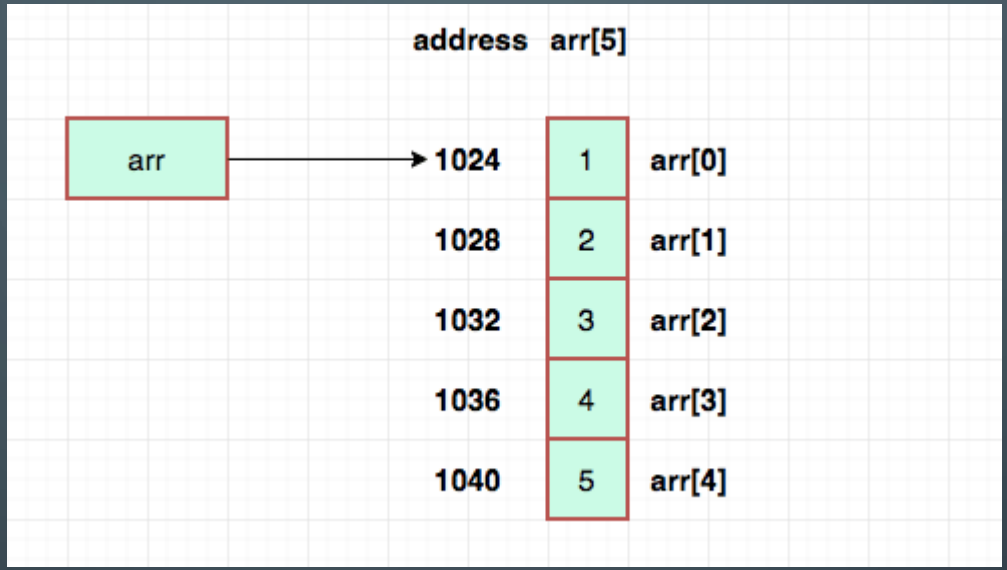


Image: https://medium.com/@akedalat/how-arrays-allocate-memory-a9bc93c0ad45

INTEGERS

- Whole number

```python
a = 6

print(a)
print(type(a))
```

```python
import numpy as np

# Manually creating an array
a = np.array([1, 2, 3])

print(a)
print(type(a))
print(type(a[0]))
```

```python
import numpy as np

# Generating a single number from 1 to 4
a = np.random.randint(1, 5)

print(a)
print(type(a))
```

```python
import numpy as np

# Generating an array of numbers from 1 to 4
a = np.random.randint(1, 5, 5)

print(a)
print(type(a))
```

FLOATS

- Irrational numbers (fractions)

```python
a = 6.0

print(a)
print(type(a))
```

```python
import numpy as np

# Manually creating an array
a = np.array([1.0, 2.0, 3.0])

print(a)
print(type(a))
print(type(a[0]))
```

```python
import numpy as np

# Generating an array of numbers from 1.0 to 4.0
a = np.linspace(1, 4, 4)

print(a)
print(type(a))
print(type(a[0]))
```

STRING

- Contains words, sentences and characters

```python
a = 'hello'

print(a)
print(type(a))

# Something strange happens here...
print(a[0])
print(type(a[0]))
```

```python
import random

# Generating a finite list of names
a = random.choices(['Leonardo', 'Michelangelo', 'Donatello', 'Raphael'], k=4)

print(a)
print(type(a))

# The data type changes here
print(a[0])
print(type(a[0]))
```

PSEUDOCODE

WHAT IS IT?

- Can be universally applied to any language
- Can use any words, but there are some cultural norms
- Basic words used to briefly describe the logic intended in a piece of code

```
# BEGIN
# Generate a finite list of 4 different names
# PRINT list
# PRINT data type of list
```

LOOPS

For and while loops can:

- Can use conditional statements – if/elif/else
- Be used to execute an iterative block of code
- Uses an ‘exit statement’ to exit the loop
  - Special case is ‘while True:’ which is used in embedded system design
    - Can also use ‘while 1 == 1:’ or ‘while 1:’

It's important to know the difference in behaviour, especially for troubleshooting

FOR LOOPS

```python
import numpy as np

a = np.array([1, 2, 3])

for i in range(0, len(a)):
    print(a[i])
```

```python
import numpy as np

a = np.array([1, 2, 3])

for i in range(len(a)):
    print(a[i])
```

```python
import numpy as np

a = np.array([1, 2, 3])

for i in range(len(a)):
    if a[i] == 1:
        a[i] *= 2
    elif a[i] == 2:
        a[i] += 4
    else:
        a[i] += a[i-1]
        # What if we change this line to a[i+1]?

print(a)
```

TRICKY, BUT STILL USEFUL FOR LOOP

```python
import numpy as np

a = np.array([1, 2, 3])

for i in a:
    print(i)
print(a)
```

```python
import numpy as np

a = np.array([1, 2, 3])

for i in a:
    if i == 1:
        i *= 2
    elif i == 2:
        i += 4
    else:
        i += i-1

    print(i)
print(a)

# Why does printing i look different to printing a?
```

FOR LOOP USING A VARIABLE FOR COUNTING

```python
import numpy as np

a = np.array([1, 2, 3])

# Use a counter to navigate indexes
counter = 0

for i in a:
    if i == 1:
        a[counter] *= 2
    elif i == 2:
        a[counter] += 4
    else:
        a[counter] += a[counter-1]

    counter += 1

    print(i)
print(a)

# Why does printing i look different to printing a?
```

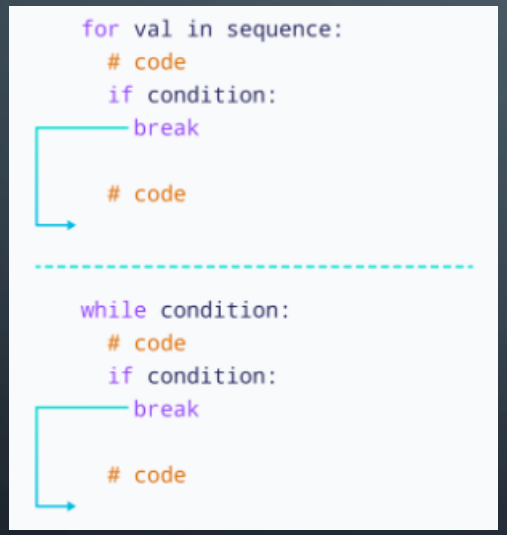
WHILE LOOPS

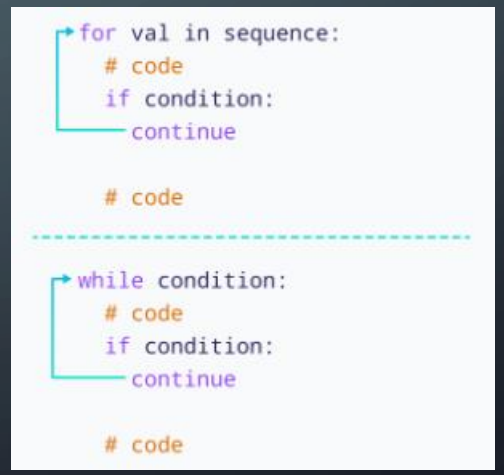
```python
i = 1
while i <= 3:
    print(i)
    i += 1
```

```python
import numpy as np

a = np.array([1, 2, 3])

i = 0
while i < len(a):
    print(a[i])
    i += 1
```

BREAKS AND CONTINUES

Sometimes under certain conditions, we want to break out of loops or skip over code and continue the iteration of loops.





Images: https://www.programiz.com/python-programming/break-continue

BREAK

```python
for i in range(1, 5):
    if i == 2:
        break
    print(i)
```

```python
i = 1
while i < 5:
    if i == 2:
        break
    print(i)
    i += 1
```

CONTINUE

```python
for i in range(1, 5):
    if i == 2:
        continue
    print(i)
```

```python
i = 1
while i < 5:
    if i == 2:
        i += 1
        continue
    print(i)
    i += 1
```

FUNCTIONS

FUNCTIONS

- Isolated recurring code that has a theme, merged together in a code block
- This allows us to reuse code without having to copy/paste multiple times
- We can use these in our main program script, although it is recommended to group these into modules and classes
- Functions make great use of data types we haven't come across yet, named 'tuple' and 'list'

DEFINING A FUNCTION

```python
v0 = 5
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)

v0 = 6
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)

v0 = 7
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)
```

CREATING THE BODY OF OUR FUNCTION

```
def heightCalculator():
    v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
```

RETURNING

```python
def heightCalculator():
    v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
    return y

v0 = 6
y = heightCalculator()
print(y)

v0 = 7
y =heightCalculator()
print(y)

# Why doesn't this work?
# How can we fix it?
```

DEFINING A FUNCTION

```python
v0 = 5
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)

v0 = 6
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)

v0 = 7
g = 9.81
t = 0.6
y = v0*t-0.5*g*t**2
print(y)
```

CREATING THE BODY OF OUR FUNCTION

```python
def heightCalculator(v0):
    # v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
```

RETURNING

```python
def heightCalculator(v0):
    #v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
    return y


v0 = 5
y = heightCalculator(v0)
print(y)

v0 = 6
y = heightCalculator(v0)
print(y)

v0 = 7
y = heightCalculator(v0)
print(y)
```

Can you think of other ways we could further optimise this code?

RETURNING

This time, what if we want to return the height at one point in time, but also our time and gravity values so the user knows the precision of the values we're using to calculate the height?

```python
def heightCalculator(v0):
    #v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
    return y, g, t


v0 = 5
y, g, t = heightCalculator(v0)
print(y)
print(g)
print(t)

v0 = 6
y, g, t = heightCalculator(v0)
print(y)
print(g)
print(t)

v0 = 7
y, g, t = heightCalculator(v0)
print(y)
print(g)
print(t)
```

```python
def heightCalculator(v0):
    #v0 = 5
    g = 9.81
    t = 0.6
    y = v0*t-0.5*g*t**2
    print(y)
    return y, g, t


v0 = 5
values = heightCalculator(v0)
print(values[0])
print(values[1])
print(values[2])

v0 = 6
values = heightCalculator(v0)
print(values[0])
print(values[1])
print(values[2])

v0 = 7
values = heightCalculator(v0)
print(values[0])
print(values[1])
print(values[2])
```

TUPLES AND LISTS

- Referenced like an array, but do not behave like an array
  - They cannot be used in vectorisation
- They can contain arrays, tuples, integers, strings, etc
  - Or a mixture of everything all in one!

MAIN DIFFERENCES BETWEEN TUPLES AND LISTS

- Tuples are immutable: This means they have a fixed size so only require a minimum amount of memory. Much like arrays, once they're created, their size cannot be changed.
- Lists are mutable: This means they're modifiable and require an extra memory block in case there is a need to extend the size of the object. Lists also have their own syntax to add and remove data. Other languages that use lists include C++, Java, C#, SQL and R.

CSV INPUT/OUTPUT

INPUT

```python
import pandas as pd

# Read in CSV
data = pd.read_csv(r'C:\Users\Student\Desktop\ENGG1003\CSVOutputExample.csv')

# Isolate columns of data by referring to their column headings
y = data['Height'].values
t = data['Time'].values
```

OUTPUT

```python
import numpy as np
import pandas as pd

v0 = 5
g = 9.81

# Initialise Arrays. Multidimensional arrays also work
t = np.linspace(0, 1, 1001)
y = v0*t-0.5*g*t**2

# Turn dictionary into Pandas DataFrame
time_dataframe = pd.DataFrame({'Time': t})
height_dataframe = pd.DataFrame({'Height': y})

# Concatenate DataFrames. axis=1 stacks vertically, axis=0 stacks horizontally
data = pd.concat([time_dataframe, height_dataframe], axis=1)

# Export to CSV
data.to_csv(r'C:\Users\Student\Desktop\ENGG1003\CSVOutputExample.csv', index=False, header=True)
```

MULTI-SCRIPT PROGRAMS

MULTI-SCRIPT PROGRAMS

- Can use modules or classes, which could also be part of a library
  - Module: A script that only contains functions and doesn't store information
  - Classes: A script containing functions and variables that can store information
  - Library: A collection of modules and/or classes

HOW TO CREATE A MODULE

```
import numpy as np

2 usages (2 dynamic)
def circumference(r):
    C = 2 * np.pi * r
    return C

def area(r):
    A = np.pi * r**2
    return A
```

HOW TO USE A MODULE

```python
import ModuleExample1a as circle

r = 3

y = circle.circumference(r)
print(y)

y = circle.area(r)
print(y)
```

THINGS TO KNOW BEFORE CREATING A CLASS

- Culturally, class names start with a capital letter. Any variables created from this object are also named with a capital letter to continue that communication
- For memory management, constructors and destructors can be used:
  - Constructors use the function '__init__()'
  - Destructors use the keyword 'del'
  - Other languages use other syntax
- Keyword 'self' is used when referring to variables and methods within the class
  - Java and C++ use the keyword 'this'
- Classes can also be used as data types

HOW TO CREATE A CLASS

```python
import numpy as np

class Circle:
    def __init__(self, r):
        self.radius = r

    def circumference(self):
        C = 2 * np.pi * self.radius
        return C

    def area(self):
        A = np.pi * self.radius**2
        return A


# Keep in mind that for other languages, the Class
# name may need to be the same as the filename.
```

```python
import numpy as np

class Circle:
    # For other languages, we would have to
    # declare the variable 'radius' to be used here.

    def circumference(self):
        C = 2 * np.pi * self.radius
        return C

    def area(self):
        A = np.pi * self.radius**2
        return A


# Keep in mind that for other languages, the Class
# name may need to be the same as the filename.
```

HOW TO USE A CLASS

```python
from ClassExample1a import Circle

r = 3
Circle1 = Circle(r)

y = Circle1.circumference()
print(y)
y = Circle1.area()
print(y)

r = 4
Circle2 = Circle(r)

y = Circle2.circumference()
print(y)
y = Circle2.area()
print(y)
```

```python
from ClassExample2a import Circle

Circle1 = Circle()
# For other languages, we declare 'radius' within the Class,
# so we can update the value of the copy here.
Circle.radius = 3

y = Circle1.circumference()
print(y)
y = Circle1.area()
print(y)

Circle2 = Circle()
Circle.radius = 4

y = Circle2.circumference()
print(y)
y = Circle2.area()
print(y)
```

HOW TO READ LIBRARIES SO YOU CAN USE THEM

FEATURES TO LOOK OUT FOR

- Class or module references in the example script
  - Class: NewClassObject = Class()
  - Module: module.function()
  - At this point, look at the class or module Python script to investigate what it expects and how it behaves
- Parameter data types passed into functions

EXAMPLES OF LIBRARIES

- Numpy: https://github.com/numpy/numpy
  - Format: https://github.com/numpy/numpy/blob/main/numpy/lib/format.py
- State Machine: https://github.com/fgmacedo/python-statemachine

KEY THINGS TO REMEMBER

- Pull out key points of an idea or brief to help construct logic and identify what kind of syntax you need
- Use pseudocode to get a rough template for the execution of your program
  - Don’t be afraid to draw pictures to help with this as well
- Use print statements for troubleshooting
  - Printing to console in embedded system design will greatly reduce the speed of your code